

Fundamentele programării

Recursivitate

O funcție recursivă : este o funcție care se auto-apelază.

```
def factorial(n):  
    """  
    compute the factorial  
    n is a positive integer  
    return n!  
    """  
    if n== 0:  
        return 1  
    return factorial(n-1)*n
```

- Recursivitate directă : P apelează P
- Recursivitate indirectă : P apelează Q, Q apelează P

Cum rezolvăm probleme folosind recursivitatea

- Definim cazul de bază : soluția cea mai simplă adică punctul unde se oprește apelul recursiv.
- Pas inductiv : împărțim problema într-o variantă mai simplă al aceleiași probleme plus ceva pași simpli,
ex: apel cu $n-1$ sau două apeluri cu $n/2$

```
def recursiveSum(l):  
    """  
    Compute the sum of numbers  
    l - list of number  
    return int, the sum of numbers  
    """  
    #base case  
    if l==[]:  
        return 0  
    #inductive step  
    return l[0]+recursiveSum(l[1:])
```

```
def fibonacci(n):  
    """  
    compute the fibonacci number  
    n - a positive integer  
    return the fibonacci number for a given n  
    """  
    #base case  
    if n==0 or n==1:  
        return 1  
    #inductive step  
    return fibonacci(n-1)+fibonacci(n-2)
```

Recursivitate în Python

- la fiecare apel de metodă se creează un nou namespace. (tabel de simboluri). Acesta conține valorile pentru parametri și variabilele locale.
- tabelul de simboluri este salvat pe stack, iar când apelul se termină tabelul se elimină din stivă.

```
def isPalindrome(str):  
    """  
    verify if a string is a palindrome  
    str - string  
    return True if the string is a palindrome False otherwise  
    """  
    dict = locals()  
    print (id(dict), dict)  
  
    if len(str)==0 or len(str)==1:  
        return True  
  
    return str[0]==str[-1] and isPalindrome(str[1:-1])
```

Recursivitate :

- avantaje :
 - claritate
 - cod mai simplu
- dezavantaje :
 - consum de memorie mai mare (pt fiecare recursiv se creează o nouă tabelă de simboluri)

Analiza complexității

Eficiența algoritmilor în raport cu:

- timpul de execuție
- spațiul de memorie

Timpul de execuție, depinde de:

- algoritmul folosit
- datele de intrare
- hardware-ul folosit
- sistemul de operare

Exemplu timp de execuție.

```
def fibonacci(n):  
    """  
    compute the fibonacci number  
    n - a positive integer  
    return the fibonacci number for a given n  
    """  
    #base case  
    if n==0 or n==1:  
        return 1  
    #inductive step  
    return fibonacci(n-1)+fibonacci(n-2)
```

```
def fibonacci2(n):  
    """  
    compute the fibonacci number  
    n - a positive integer  
    return the fibonacci number for a given n  
    """  
    sum1 = 1  
    sum2 = 1  
    rez = 0  
    for i in range(2, n+1):  
        rez = sum1+sum2  
        sum1 = sum2  
        sum2 = rez  
    return rez
```

```
def measureFibo(nr):  
    sw = Stopwatch()  
    print "fibonacci2(", nr, ") =", fibonacci2(nr)  
    print "fibonacci2 take " +str(sw.stop())+" seconds"  
  
    sw = Stopwatch()  
    print "fibonacci(", nr, ") =", fibonacci(nr)  
    print "fibonacci take " +str(sw.stop())+" seconds"
```

measureFibo(32)

```
fibonacci2( 32 ) = 3524578  
fibonacci2 take 0.0 seconds  
fibonacci( 32 ) = 3524578  
fibonacci take 1.7610001564 seconds
```

Eficiența algoritmilor

Este definită ca fiind cantitatea de resurse utilizate de algoritm
(timp, memorie)

Măsurarea eficienței:

- analiză matematică (asimptotică) a algoritmului:

descrie eficiența sub forma unei funcții matematice
estimează timpul de execuție pentru toate intrările posibile

- analiză empirică a algoritmului:

determină timpul exact de execuție pentru date specifice
nu se poate prezice timpul pentru toate datele de intrare

Timpul de execuție este studiat în relație cu dimensiunea datelor de intrare.

- Estimăm timpul de execuție în funcție de dimensiunea datelor.
- Realizăm o analiză asimptotică și determinăm ordinul de mărime pentru resursa utilizată (timp, memorie)

Complexitate

- **caz favorabil** - datele de intrare conduc la timp de execuție minim

$$BC(A) = \min_{I \in D} E(I)$$

- **caz defavorabil** - datele de intrare conduc la timp de execuție maxim

$$WC(A) = \max_{I \in D} E(I)$$

- **caz mediu** - timp de execuție

(average complexity) $AC(A) = \sum_{I \in D} P(I) E(I)$

A - algoritmul, $E(I)$ - numărul de operații,
 $P(I)$ - probabilitatea de a avea I ca și date de intrare
 D - mulțimea tuturor datelor de intrare

Complexitate timp de execuție

- numărăm pași (nr de instrucțiuni, comparații, adunări, etc.)
- numărul de pași nu este un număr fixat, este o funcție $T(n)$, este în funcție de dimensiunea datelor n , nu rezultă timpul exact de execuție
- se surprinde doar esențialul: cum crește timpul de execuție în funcție de dimensiunea datelor

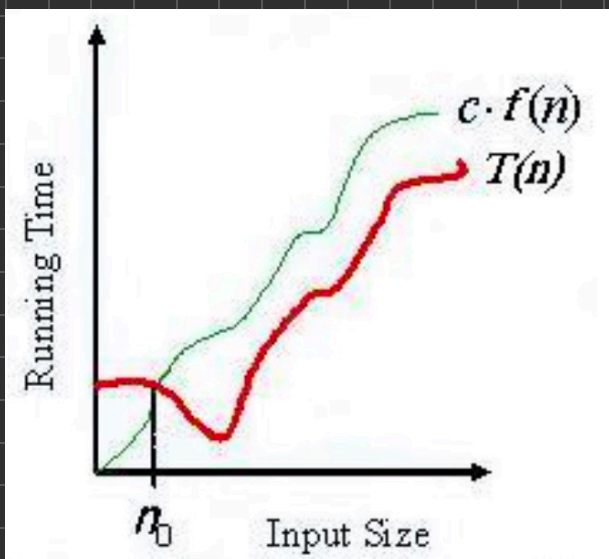
• putem ignora constante mici

$$\text{Ex. } T(n) = 13 \cdot n^3 + 42 \cdot n^2 + 2 \cdot n \cdot \log_2 n + 3 \cdot \sqrt{n}$$

$0 < \log_2 n < n$, $\sqrt{n} < n$ $\forall n > 1 \Rightarrow n^3$ domină expresia când n este mare, ca urmare timpul de execuție crește cu ordinul lui n^3 , notăție: $T(n) \in O(n^3)$ și se citește $T(n)$ este de ordinul n^3

În continuare, avem o funcție $f: \mathbb{N} \rightarrow \mathbb{R}$ și $T: \mathbb{N} \rightarrow \mathbb{N}$ care dă complexitatea timp de execuție a unui algoritmul.

Def: (Notatia O , "Big-oh"). Spunem că $T(n) \in O(f(n))$ dacă există c și n_0 constante pozitive a.i. $0 \leq T(n) \leq c \cdot f(n)$, $\forall n \geq n_0$.



Notăția O dă marginea superioară

Def alternativă: Spunem că: $T(n) \in O(f(n))$ dacă $\exists$
 $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} < +\infty$

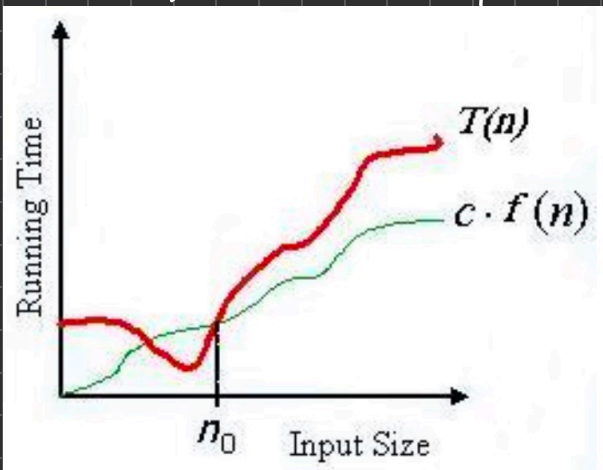
Obs:

• Dacă $T(n) = 13 \cdot n^3 + 42 \cdot n^2 + 2 \cdot n \cdot \log_2 n + 3 \cdot \sqrt{n}$, atunci $\lim_{n \rightarrow \infty} \frac{T(n)}{n^3} = 13$

Deci putem spune că $T(n) \in O(n^3)$

• Notatia O este bună pentru a da o limită superioară a funcției. Totuși, dacă $T(n) \in O(n^3)$, atunci este și $O(n^4)$, $O(n^5)$, etc. ptc limita este O . Din această cauză avem nevoie de o notăție pentru limita inferioară a complexității, pe care o notăm cu Ω .

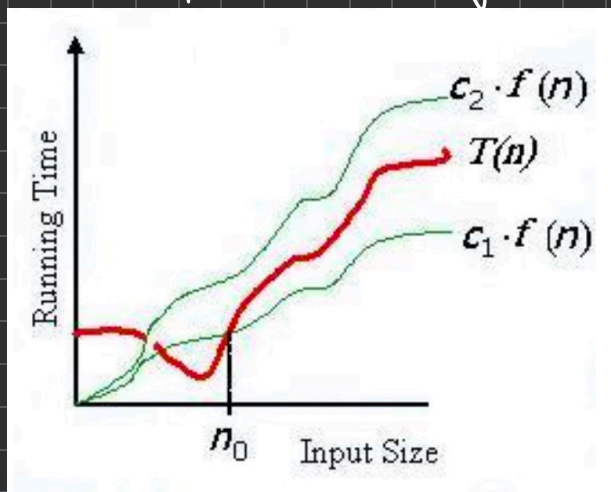
Def (Notăția Ω , "Big-omega"): Spunem că $T(n) \in \Omega(f(n))$ dacă există c și n_0 constante pozitive c.î. $0 \leq c \cdot f(n) \leq T(n)$, $\forall n \geq n_0$



notația Ω de marginea inferioară

Def alternativă: Spunem că: $T(n) \in \Omega(f(n))$ dacă $\exists$
 $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} > 0$

Def (Notația Θ , "Big-Theta"). Spunem că $T(n) \in \Theta(f(n))$ dacă
 $T(n) \in O(f(n))$ și $T(n) \in \Omega(f(n))$, astfel spus, există c_1, c_2 și n_0
constante pozitive c.î. $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$, $\forall n \geq n_0$.



notația Θ mărginește o funcție până la factor constanți

Def alternativă Spunem că: $T(n) \in \Theta(f(n))$ dacă $\exists$

$$\lim_{n \rightarrow \infty} 0 < \frac{T(n)}{f(n)} < +\infty$$

Obs:

1. Timpul de execuție este $\Theta(f(n))$ dacă timpul algoritmului de execuție este $O(f(n))$ în cazul cel mai defavorabil și $\Omega(f(n))$ în cazul cel mai favorabil.
2. Notatia $O(f(n))$ este de cele mai multe ori folosită în locul notatiei $\Theta(f(n))$
3. Dacă $T(n) = 13 \cdot n^3 + 42 \cdot n^2 + 2 \cdot n \cdot \log_2 n + 3\sqrt{n}$ atunci $\lim_{n \rightarrow \infty} \frac{T(n)}{n^3} = 13$, deci $T(n) \in \Theta(n^3)$. Acest lucru este dedus din $T(n) \in O(n^3)$ și $T(n) \in \Omega(n^3)$.

Suma

for i in range $(0, n)$:
instructions.

Presupunând că ceea ce este în corpul structurii repetitive se execută în $f(i)$ pași, timpul de execuție poate fi estimat astfel $T(n) = \sum_{i=1}^n f(i)$

Example in sumu:

```
def f1(n):
    s = 0
    for i in range(1,n+1):
        s=s+i
    return s
```

$$T(n) = \sum_{(i=1)}^n 1 = n \rightarrow T(n) \in \theta(n)$$

Complexitate (Overall complexity) $\theta(n)$

Cazurile Favorabil/Mediu/Defavorabil sunt identice

```
def f2(n):
    i = 0
    while i<=n:
        #atomic operation
        i = i + 1
```

$$T(n) = \sum_{(i=1)}^n 1 = n \rightarrow T(n) \in \theta(n)$$

Overall complexity $\theta(n)$

Cazurile Favorabil/Mediu/Defavorabil sunt identice

```
def f3(l):
    """
    l - list of numbers
    return True if the list contains
    an even nr
    """
    poz = 0
    while poz<len(l) and l[poz]%2 !=0:
        poz = poz+1
    return poz<len(l)
```

Caz favorabil:

primul element e număr par: $T(n) = 1 \in \theta(1)$

Caz defavorabil:

Nu avem numere pare în listă: $T(n) = n \in \theta(n)$

Caz mediu:

While poate fi executat 1,2,..n ori (același probabilitate).

Numărul de pași = numărul mediu de iterații

$$T(n) = (1 + 2 + \dots + n)/n = (n + 1)/2 \rightarrow T(n) \in \theta(n)$$

Complexitate $O(n)$

```
def f4(n):
    for i in range(1,2*n-2):
        for j in range(i+2,2*n):
            #some computation
            pass
```

$$T(n) = \sum_{(i=1)}^{(2n-2)} \sum_{(j=i+2)}^{2n} 1 = \sum_{(i=1)}^{(2n-2)} (2n - i - 1)$$

$$T(n) = \sum_{(i=1)}^{(2n-2)} 2n - \sum_{(i=1)}^{(2n-2)} i - \sum_{(i=1)}^{(2n-2)} 1$$

$$T(n) = 2n \sum_{(i=1)}^{(2n-2)} 1 - (2n - 2)(2n - 1)/2 - (2n - 2)$$

...

$$T(n) = 2n^2 - 3n + 1 \in \theta(n^2) \quad \text{Overall complexity } \theta(n^2)$$

```
def f5():
    for i in range(1,2*n-2):
        j = i+1
        cond = True
        while j<2*n and cond:
            #elementary operation
            if someCond:
                cond = False
```

Caz favorabil: While se execută odată $T(n) = \sum_{(i=1)}^{(2n-2)} 1 = 2n - 2 \in \theta(n)$

Caz defavorabil: While executat $2n - (i + 1)$ ori

$$T(n) = \sum_{(i=1)}^{(2n-2)} (2n - i - 1) = \dots = 2n^2 - 3n + 1 \in \theta(n^2)$$

Caz mediu: Pentru un i fixat While poate fi executat 1,2..2n-i-1 ori număr mediu de pași: $C_i = (1 + 2 + \dots + 2n - i - 1)/2n - i - 1 = \dots = (2n - i)/2$

$$T(n) = \sum_{(i=1)}^{(2n-2)} C_i = \sum_{(i=1)}^{(2n-2)} (2n - i)/2 = \dots \in \theta(n^2)$$

Overall complexity $O(n^2)$

Formule cu sume.

$$\sum_{i=1}^n 1 = n$$

suma constantă

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

suma liniară

$$\sum_{i=1}^{n^2} i^2 = \frac{n(n+1)(2n+1)}{6}$$

suma pătratică

$$\sum_{i=1}^n \frac{1}{i} = \ln(n) + O(1)$$

suma armonică

$$\sum_{i=1}^n c^i = \frac{c^{n+1} - 1}{c - 1}, c \neq 1$$

progresia geometrică

Complexități uzuale

$T(n) \in O(1)$ - timp constant

$T(n) \in O(\log_2 \log_2(n))$ - timp foarte rapid

$T(n) \in O(\log_2 n)$ - complexitate logaritmică

$T(n) \in O((\log_2 n)^k)$ - complexitate polilogaritmică

$T(n) \in O(n)$ - complexitate liniară

$T(n) \in O(n \log_2 n)$ - (Merge Sort, Quick Sort)

$T(n) \in O(n^2)$ - complexitate pătratică

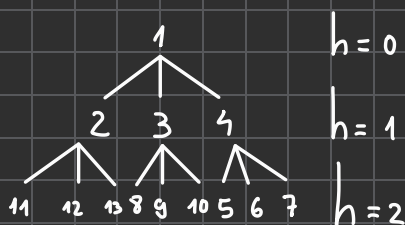
$T(n) \in O(n^k)$ - complexitate polinomială

$T(n) \in O(2^n), O(n^3), O(n!)$ - complexitate exponențială

Recurență

O recurență este o formulă matematică definită recursiv.

Ex: numărul de noduri total dintr-un arbore cu 3 noduri complet de înălțime h poate să fie descris folosind o formulă de recurență astfel:



$$\begin{cases} N(0) = 1 \end{cases}$$

$$\begin{cases} N(h) = 3 \cdot N(h-1) + 1, & h \geq 1 \end{cases}$$

$$\Rightarrow N(h) = 3^0 + 3^1 + 3^2 + \dots + 3^h = \sum_{i=0}^h 3^i$$

Example:

<pre>def recursiveSum(l): """Compute the sum of numbers l - list of number return the sum of numbers """ #base case if l==[]: return 0 #inductive step return l[0]+recursiveSum(l[1:])</pre>	Recurrence: $T(n) = \begin{cases} 1 & \text{if } n = 0 \\ T(n-1) + 1 & \text{otherwise} \end{cases}$ $T(n) = T(n-1) + 1$ $T(n-1) = T(n-2) + 1$ $T(n-2) = T(n-3) + 1 \Rightarrow T(n) = n + 1 \in \theta(n)$ $\dots = \dots$ $T(1) = T(0) + 1$
<pre>def hanoi(n, x, y, z): """ n - number of disk on the x stick x - source stick y - destination stick z - intermediate stick """ if n==1: print ("disk 1 from",x,"to",y) return hanoi(n-1, x, z, y) print ("disk ",n,"from",x,"to",y) hanoi(n-1, z, y, x)</pre>	Recurrence: $T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n-1) + 1 & \text{otherwise} \end{cases}$ $T(n) = 2T(n-1) + 1$ $T(n-1) = 2T(n-2) + 1 =$ $T(n-2) = 2T(n-3) + 1$ $\dots = \dots$ $T(1) = T(0) + 1$ $T(n) = 2T(n-1) + 1$ $2T(n-1) = 2^2T(n-2) + 2$ $2^2T(n-2) = 2^3T(n-3) + 2^2$ $\dots = \dots$ $2^{(n-2)}T(2) = 2^{(n-1)}T(1) + 2^{(n-2)}$ $T(n) = 2^{(n-1)} + 1 + 2 + 2^2 + 2^3 + \dots + 2^{(n-2)}$ $T(n) = 2^n - 1 \in \theta(2^n)$

Complexitatea spatiu de memorie

Estimare cantitatea de memorie necesara algoritmului: pentru stocarea datelor de intrare, a rezultatelor finale si cele intermediare.

Se estimeaza ca si timpul de executie, in notatiile O, θ, Ω

Exemplu:

<pre>def iterativeSum(l): """ Compute the sum of numbers l - list of number return int, the sum of numbers """ rez = 0 for nr in l: rez = rez+nr return rez</pre>	Avem nevoie de spatiu pentru numerele din lista $T(n) = n \in \theta(n)$
<pre>def recursiveSum(l): """ Compute the sum of numbers l - list of number return int, the sum of numbers """ #base case if l==[]: return 0 #inductive step return l[0]+recursiveSum(l[1:])</pre>	Recurență: $T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(n-1) + 1 & \text{otherwise} \end{cases}$

Analiza complexității (timp / spațiu) pentru o funcție

1. Dacă există caz favorabil / defavorabil.

- descrie BC
- calculează complexitatea pentru BC
- descrie WC
- calculează complexitatea pentru WC
- calculează complexitatea medie
- calculează complexitatea generală

2. Dacă $BC = WC = AC$

- calculează complexitatea

Calculul complexității:

- dacă avem recurență calculăm folosind egalitatea
- altfel calculăm folosind sumele

Exemplu:

```
for i in range(1, n):
```

```
    for j in range(i, n):
```

```
        count += 1
```

$$\sum_{i=1}^{n-1} \sum_{j=i}^{n-1} 1 = \sum_{i=1}^{n-1} n-i = \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i = n(n-1) - \frac{n(n-1)}{2} = \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2} \in \Theta(n^2)$$